

VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI
CAMPUS

INNOVATIVE DESIGN PROJECT | BACSE291

Review Report II

Crypto-Agility Navigator: Dataflow-Aware Cryptographic Inventory and Prioritised Post- Quantum Migration

Submitted by

	Name	Registration No.
1	Avi Dhandhania	25BCE1207
2	Anmol Saluja	25BCE1332
3	Avika Tyagi	25BCE1294

Project guide

Dr. E INIYA NEHRU

54128

SCOPE

Acknowledgement

We express our sincere gratitude to our project guide, **Dr. E INIYA NEHRU** (54128), SCOPE, for the constant guidance, encouragement and constructive feedback that shaped every stage of this work. Their insight into the problem of cryptographic inventory and post-quantum migration helped us sharpen the research question behind the *Crypto-Agility Navigator*, and their patience during the design discussions and prototype reviews was invaluable.

We are equally thankful to the Review I evaluators and to the panel members for their observations, which directly informed the system design and the scope of the initial prototype presented in this report. We also thank the BACSE291 course coordinators and the faculty of the school for providing the framework, schedule and resources that made this project possible.

Finally, we thank our families and friends for their support and understanding throughout this project.

Date: _____

Guide's Signature
DR. E INIYA NEHRU
54128
SCOPE

Table of Contents

1. Executive Summary and Problem Identification.....	1
1.1 The Technical Problem and Threat Model	1
1.2 The Specific Gap in State-of-the-Art Tooling.....	1
1.3 Formalisation through Mosca’s Inequality.....	1
1.4 Regulatory Justification and Compliance Urgency	2
2. Comprehensive Literature Survey and Critical Gap Analysis.....	2
2.1 Survey Methodology	2
2.2 Summary of Surveyed Work: Challenges and Research Gaps (P1–P16).....	3
2.3 Critical Analysis: What Prior Work Gets Wrong, and Our Corrections	4
2.4 Consolidated Research Gap Matrix.....	5
3. Project Objectives, Scope and Novelty	6
3.1 Primary and Secondary Objectives	6
3.2 The Five Pillars of Novelty	6
4. Requirement Analysis and Problem Understanding.....	6
4.1 Stakeholder Analysis and Target Ecosystems	6
4.2 Functional Requirements (FR)	7
4.3 Non-Functional Requirements (NFR)	7
4.4 Regulatory and Compliance Constraints	7
5. System Design and Architecture	8
5.1 End-to-End Architectural Pipeline	8
5.2 Subsystem 1 — Multi-Language Cryptographic Discovery Layer	8
5.3 Subsystem 2 — Inter-Procedural Semantic Dataflow and Retention Engine	8
5.4 Subsystem 3 — HNDL Prioritisation, Scoring and Noise Suppression.....	9
5.5 Subsystem 4 — Hybrid Patch Synthesis and Differential Verification.....	9
5.6 Data Schema and CycloneDX 1.6 CBOM Extension	9
6. Component and Tool Selection with Technical Justification	9
6.1 Parsing and AST Infrastructure	9
6.2 Static Analysis and Taint Tracking	10
6.3 Post-Quantum Cryptographic Library	10
6.4 CBOM Schema.....	10
6.5 Patch Verification Strategy.....	10
7. Initial Prototype and Module Development.....	11
7.1 Implemented Package Structure	11
7.2 Implemented Modules.....	11
7.3 Empirical Validation on the Benchmark Example	11
7.4 Automated Unit Test Suite	14
7.5 Interactive Web Dashboard and REST API	15
7.6 TRL 3 Milestone Evidence Summary	16
8. Innovation and Feasibility	16
8.1 Mathematical Basis of the HNDL Score	16
8.2 Soundness versus Completeness	16
8.3 Computational and Memory Feasibility	16
9. Project Planning, Teamwork and Presentation.....	17
9.1 Work Division Principle.....	17
9.2 Work Division for Review II (Completed).....	17
9.3 Work Plan for Future Reviews	18
10. References and Regulatory Standards.....	18
10.1 Academic References	18
10.2 Regulatory and Standards References	19

1. Executive Summary and Problem Identification

1.1 The Technical Problem and Threat Model

Every enterprise software system depends on cryptography it cannot reliably enumerate or sequence for migration. A typical mid-sized banking or healthcare application invokes RSA, ECDSA, AES and SHA-family primitives from a complex mixture of first-party business logic, third-party packages, container base images, TLS terminators, database drivers and infrastructure-as-code (IaC) — with no single machine-readable artefact recording where those invocations exist or what data assets they protect.

Peter Shor’s polynomial-time quantum algorithm shows that integer factorisation and discrete-logarithm problems are solvable on a Cryptographically Relevant Quantum Computer (CRQC), rendering classical public-key cryptography (RSA, ECDSA, ECDH, DSA, Ed25519) insecure. The National Institute of Standards and Technology (NIST) has standardised the post-quantum replacements:

- **FIPS 203** — Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM, derived from CRYSTALS-Kyber).
- **FIPS 204** — Module-Lattice-Based Digital Signature Algorithm (ML-DSA, derived from CRYSTALS-Dilithium).
- **FIPS 205** — Stateless Hash-Based Digital Signature Algorithm (SLH-DSA, derived from SPHINCS+).

Consequently, *what* to migrate to is settled. The unsolved engineering challenge is industrial rather than mathematical: **an organisation cannot migrate what it cannot locate, and cannot sequence a multi-year migration without knowing which of its thousands of cryptographic call sites are actually dangerous.**

The urgency is set by the **Harvest-Now-Decrypt-Later (HNDL)** threat model. A passive adversary intercepts and stores encrypted network traffic and database backups today at negligible storage cost, intending to decrypt the captured ciphertext once a CRQC becomes operational. Detection after the fact provides no remedy, and forward secrecy cannot protect data retroactively.

1.2 The Specific Gap in State-of-the-Art Tooling

Cryptographic inventory scanners have recently emerged — including CBOMkit, IBM Quantum Safe Explorer, SandboxAQ AQtive Guard and the academic scanner Crypsy [P1] — standardising around the **Cryptography Bill of Materials (CBOM)**, an extension to CycloneDX 1.6 [P6]. However, the output of all existing tools is **semantically flat**: a list of the form “*RSA-2048 appears at payments/crypto.py:214*”. Two measured operational consequences follow:

1. **Findings are not actionable (noise).** Existing scanners fire pattern-matching rules on every matching invocation regardless of usage context. Näther and Hirsch report that on real deployed services Crypsy achieves a real-world actionable precision of approximately **0.30** [P1]: over two-thirds of reported findings are noise, predominantly non-security hashing (cache keys, ETags, checksums) matched by the same heuristic rules as password hashing.
2. **Findings are not ordered (flat scoring).** Where risk scoring exists it is *algorithm-intrinsic*. Shaw’s quantum-aware scorer [P2] derives a 0–10 severity from key size, Shor-path qubit cost, Grover speed-up and forward-security exposure. Every RSA-2048

call site in a codebase therefore receives an identical score, which gives no signal for prioritising an engineering remediation backlog.

1.3 Formalisation through Mosca's Inequality

The migration deadline is formalised by **Mosca's inequality**. For a given data path, let

- x = data confidentiality lifetime (years the data must remain secret);
- y = migration duration (years required to re-engineer the system);
- z = CRQC operational horizon (years until a cryptographically relevant quantum computer exists).

A data path is **already breached** today if

$$x + y > z$$

Critically, x is a property of the *data being protected*, not of the cipher.

Worked example: two identical call sites in one repository

```
# Site A — payments/archive.py
record = build_settlement_record(txn)      # source: persistent database read
blob = rsa_oaep_encrypt(record, archive_pubkey) # RSA-2048 OAEP
s3.put_object(Bucket="settlements-archive", # sink: cloud object store,
              Key=k, Body=blob)             # lifecycle: retain 10 years

# Site B — web/session.py
tok = make_csrf_token()                    # source: local in-memory RNG
sealed = rsa_oaep_encrypt(tok, session_pubkey) # RSA-2048 OAEP (identical primitive)
redis.setex(k, 900, sealed)                 # sink: cache, TTL 900 seconds
```

- **Site A** protects a statutory financial settlement record with a 10-year retention rule in multi-tenant cloud storage. With $x = 10$, $y = 2$ and $z = 7$ (aligned with the national 2027–2029 / 2033 targets), $10 + 2 > 7$: **Site A is already breached under Mosca's inequality**.
- **Site B** protects an ephemeral CSRF token that expires in 15 minutes ($x \approx 0.00003$ years). Migrating Site B delivers no risk reduction.

Existing discovery tools and algorithm-intrinsic scorers report both sites identically as RSA-2048 with identical severity. Yet the correct engineering order is unambiguous, and every piece of evidence needed to derive it — `s3.put_object` with 10-year retention versus `redis.setex` with a 900-second TTL — is present in the repository source code and configuration.

1.4 Regulatory Justification and Compliance Urgency

1. **India Critical Information Infrastructure (CII) deadline (2027–2029).** The National Quantum-Safe Task Force has set a migration timeline for critical national infrastructure.
2. **Reserve Bank of India (RBI) Q-SAFE Committee.** Formed under IIT Madras leadership, it directs scheduled commercial banks and payment operators to build cryptographic inventories and assess crypto-agility.
3. **SEBI Cyber Security and Cyber Resilience Framework (CSCRF).** Requires registered intermediaries to identify and remediate HNDL attack surfaces.
4. **US Executive Order 14412 (June 2026).** Instructs CISA and NIST to standardise the minimum elements of a Cryptography Bill of Materials within 270 days.
5. **NIST final standards.** FIPS 203, 204 and 205 establish post-quantum primitives for production systems.

2. Comprehensive Literature Survey and Critical Gap Analysis

2.1 Survey Methodology

A survey was conducted across ACM Digital Library, IEEE Xplore, IACR Cryptology ePrint, arXiv and Springer, covering January 2024 to August 2026, using combinations of the terms *crypto-agility*, *cryptographic inventory*, *CBOM*, *PQC migration*, *static analysis*, *HNDL* and *Mosca inequality*. Forward and backward citation chasing from the most recent scanners identified **16 core papers**: eleven published in 2025–2026, three from 2024, and two foundational classical baselines (2017, 2019).

2.2 Summary of Surveyed Work: Challenges and Research Gaps (P1–P16)

Table 2.1: Literature analysis (P1–P8)

Citation / Publication	Core methodology / focus	Key challenges identified	Unresolved research gaps	Future research direction
Näther & Hirsch (2026) Crypsy [P1] <i>arXiv:2608.04857</i>	214-rule static scanner with rule repository and CBOM export for Go and Python.	Rules fire on every match; real-world actionable precision ≈ 0.30 . Recall 0.66 because runtime-computed arguments are missed.	No dataflow to the protected data; no prioritisation of findings.	Layer inter-procedural semantic dataflow and HNDL scoring on top of rule-based discovery (N1–N3).
Shaw (2026) Quantum-Safe Code Auditing [P2] <i>arXiv:2604.00560</i>	Regex detection of 15 cipher classes, LLM enrichment and a VQE-based quantum-threat score (0–10).	Score depends only on the algorithm, so every RSA-2048 site scores the same. Evaluated on a 10.4% sample with 100% recall.	No data-path context (retention, exposure); quantum scoring does not forecast real qubit needs.	Score data paths by retention $\times$ exposure $\times$ algorithm $\times$ key reuse; evaluate with nDCG and Kendall- τ (N2).
Pallarés de Bonrostro et al. (2026) LLM-based migration [P3] <i>arXiv:2606.07341</i>	Evaluated LLMs on migrating 800 paired synthetic Python fragments across 6 cryptographic families.	Fine-tuned model reached 92.5% correctness (zero-shot 15%); degrades on multi-file repositories.	Round-trip tests can pass while parameters are silently downgraded; vulnerable fragments assumed already located.	Template-constrained hybrid patches gated by differential, property and downgrade checks (N4).
Zhang (2026) AQuA vision [P4] ICSE 2026, <i>arXiv:2602.05759</i>	Quantum-safe software-engineering agenda: PQC-aware detection, semantic refactoring, hybrid verification.	Two-page vision paper; the agenda is not turned into a working tool.	No implementation and no evaluation.	Implement and evaluate the agenda end to end; this project delivers a working prototype.
Costa (2026) CARS [P5] IACR ePrint 2026/1467	Delphi-derived five-dimension crypto-agility readiness score.	One composite index per repository (mean 24.9–47.5 over 43 repositories) is not actionable for sprint planning.	Subjective expert weights never validated against real migration outcomes.	Score individual data paths and validate against expert-adjudicated priority orderings.
IBM Research (2026) CBOM anatomy [P6] <i>Eurocrypt 2026</i>	Object model for cryptographic assets, dependencies and evidence in CycloneDX.	Describes which algorithms exist, not which data they protect or for how long.	No fields for data lifetime, exposure surface or HNDL risk.	Extend the CBOM with dataflowProperties (source, sink, retention, HNDL risk).
CBOM Compliance (2026) Policy engine [P7] <i>Springer LNCS</i>	Policy-driven engine classifying CBOM assets against machine-readable compliance rules.	Operates on flat CBOMs, so its verdicts are only as rich as the inventory it receives.	No retention- or dataflow-aware input.	Feed the policy layer with the enriched, dataflow-aware CBOM.

Citation / Publication	Core methodology / focus	Key challenges identified	Unresolved research gaps	Future research direction
Practical Feasibility of HNDL (2026) HNDL analysis [P8] <i>arXiv:2603.01091</i>	Economic and practical feasibility of harvest-now-decrypt-later, including decay of data sensitivity.	For 25–50 year healthcare retention the Mosca deadline has already passed; forward secrecy cannot help retroactively.	Analysis stays at the data-class level; no tooling locates the affected code.	Bind retention lifetimes to individual call sites (N1).

Table 2.2: Literature analysis (P9–P16)

Citation / Publication	Core methodology / focus	Key challenges identified	Unresolved research gaps	Future research direction
P9, P10 (2025–2026) PQC surveys <i>arXiv:2510.10436</i> ; <i>arXiv:2603.06969</i>	Consolidated post-FIPS survey and strategic roadmap for enterprises.	Consensus on hybrid deployment (e.g. X25519 + ML-KEM), but guidance is high level.	No automated inventory-to-prioritisation tooling.	Operationalise the roadmap as one pipeline: discovery, prioritisation and verification.
Migration of Executables (2024) Binary toolchain [P11] <i>arXiv:2409.07852</i>	Binary-level disassembly and migration toolchain for compiled software.	Binary analysis loses variable names, types and configuration context; predates the NIST standards.	No source-level semantics or retention evidence.	Source-level analysis (binary analysis stays out of scope and complementary).
Cost of Waiting (2026) Decision theory [P12] <i>Frontiers Quantum Sci.</i>	Decision-theoretic model of early versus late PQC migration under CRQC-arrival uncertainty.	Modelled at organisation level, not for concrete code locations.	Not connected to per-call-site data.	Use the utility model as the basis for per-path exposure scoring.
Rahaman et al. (2019) CryptoGuard [P13] <i>ACM CCS 2019</i>	Backward inter-procedural dataflow detecting 22 classical crypto-API misuse patterns in Java.	Targets classical misuse, not post-quantum readiness.	No forward tracing to sinks or retention; no PQC awareness.	Extend inter-procedural dataflow (both directions) to HNDL-aware analysis.
Näther et al. (2024) Migration SLR [P14] <i>arXiv:2404.12854</i>	Systematic review defining four migration phases: inventory, prioritisation, migration, verification.	Lacks formal definitions; implementations are “mostly experimental”.	No validated tooling covering all four phases.	Formalise Mosca’s inequality and deliver one pipeline from inventory to verification.
Federal Reserve FEDS (2025) HNDL and banking [P15]	Financial-stability analysis of HNDL risk for regulated banking institutions.	HNDL is a systemic concern for supervisors, but no engineering method follows from it.	No way to turn supervisory concern into a code-level backlog.	Link regulatory requirements to a ranked remediation backlog.
Krüger et al. (2017) CogniCrypt [P16] <i>IEEE/ACM ASE 2017</i>	Developer-facing generation of correct cryptographic code and misuse analysis.	Rule-based synthesis; superseded on misuse detection by modern LLMs.	Not designed for hybrid PQC migration or prioritisation.	Verified hybrid templates for PQC replacement.

2.3 Critical Analysis: What Prior Work Gets Wrong, and Our Corrections

Critique of P1 — Crypsy (closest state-of-the-art baseline)

- *Flaw 1 — context-blindness.* Rules fire on every matching call. The authors concede real-world actionable precision of only ≈ 0.30 ; a hashing rule cannot tell a password check from a web-cache key.
- *Flaw 2 — no dataflow.* Crypsy has no notion of what data enters or leaves a primitive.
- *Flaw 3 — no prioritisation.* Hundreds of findings are reported as an unordered flat set, leaving engineers without a roadmap.
- *Flaw 4 — runtime-argument blindness.* Precision 0.87 versus recall 0.66: about one-third of assets are missed because arguments computed at runtime cannot be resolved.

Our correction. We adopt Crypsy’s rule-corpus concept as an *input*, not a competitor, and layer **inter-procedural semantic dataflow** over the discovery stage. Dataflow resolves Flaw 1 (password versus cache key) and Flaw 2 by construction, and Flaw 3 through an HNDL Exposure Score. Flaw 4 is addressed by constant propagation and local type inference.

Critique of P2 — Quantum-Safe Code Auditing (Shaw)

- *Flaw 1 — the score cannot discriminate.* The VQE threat score depends only on algorithm parameters, so every RSA-2048 site receives an identical value.
- *Flaw 2 — decorative quantum machinery.* A parameterised 2-qubit Hamiltonian is used to compute a weighted sum whose coefficients are published constants; the authors admit the output is not a forecast of real qubit requirements.
- *Flaw 3 — evaluation on an easy sample.* A 10.4% stratified sample with 100% recall suggests an artificially simple detection task.

Our correction. Our score is a function of the **protected data path** (retention lifetime $\times$ exposure surface $\times$ algorithm $\times$ key reuse), and is evaluated on an annotated benchmark using ranking metrics (nDCG@20, Kendall- τ) that P2 never reports.

Critique of P3 — LLM-based migration of code fragments

- *Flaw 1 — synthetic single-fragment ceiling.* 800 synthetic snippets; performance drops sharply with multi-file dependencies.
- *Flaw 2 — superficial verification.* Correctness is judged by an encrypt/decrypt round trip, which still passes if parameters are silently downgraded or interoperability with unmigrated peers is broken.
- *Flaw 3 — no discovery step.* The vulnerable fragment is assumed to have been located by a human.

Our correction. Patch synthesis is the terminal stage of our pipeline, operating on sites located by our own dataflow engine in multi-file repositories. Patches are generated from **verified hybrid templates** and gated by differential equivalence, property-based tests and downgrade-resilience checks.

Critique of P5 — CARS

CARS scores a whole repository with one composite index (wrong granularity for a two-week sprint) using Delphi-derived weights never validated against real migration outcomes.

Our correction: we score individual data paths and validate against expert-adjudicated priority orderings on held-out repositories.

2.4 Consolidated Research Gap Matrix

Capability	P1	P2	P3	P5	P6/P7	P8/P12	P13	Ours
Multi-language discovery	✓	●	✗	●	✗	✗	●	✓
CycloneDX 1.6 CBOM output	✓	✗	✗	✗	✓	✗	✗	✓
Dataflow to protected data (N1)	✗	✗	✗	✗	✗	✗	●	✓
Retention-lifetime inference (N1)	✗	✗	✗	✗	✗	manual	✗	✓
Call-site-varying risk score (N2)	✗	✗	✗	✗	✗	✗	✗	✓
Ranking evaluated with nDCG / τ (N2)	✗	✗	✗	✗	✗	✗	✗	✓
Context-based noise suppression (N3)	✗	●	✗	✗	●	✗	✗	✓
Differentially verified hybrid patches (N4)	✗	✗	●	✗	✗	✗	✗	✓
Public annotated priority benchmark (N5)	✗	✗	✗	✗	✗	✗	✗	✓

(✓ fully present · ● partial / limited · ✗ absent)

3. Project Objectives, Scope and Novelty

3.1 Primary and Secondary Objectives

ID	Objective	Measurable target
O1	Multi-language discovery. AST-based cryptographic discovery engine emitting standard CycloneDX 1.6 CBOM.	$F_1 \geq 0.85$ (Crypsy 0.75; CBOMkit 0.66).
O2	Semantic dataflow binding. Inter-procedural analysis binding each call site to its plaintext source, ciphertext sink and retention evidence.	$\geq 70\%$ call-site binding; $\geq 50\%$ retention extraction.
O3	HNDL exposure scoring and prioritisation. Data-path-dependent score operationalising Mosca’s inequality.	$\text{nDCG@20} \geq 0.80$; $\text{Kendall-}\tau \geq 0.60$ against expert order.
O4	Noise suppression. Filter non-security cryptographic operations using dataflow context.	Actionable precision ≥ 0.70 (baseline ≈ 0.30) at ≤ 5 points recall loss.
O5	Verified hybrid patch synthesis. Hybrid PQC patches (X25519 + ML-KEM) verified by differential testing before pull-request emission.	$\geq 80\%$ pass rate on real multi-file repositories.
O6	Public benchmark release. Retention-annotated cryptographic benchmark with expert priority orders.	15–20 real-world repositories.
O7	Evaluation methodology (secondary). Standardise the ranking-evaluation protocol for cryptographic migration tooling.	Documented, reproducible protocol (nDCG@20, Kendall- τ).

3.2 The Five Pillars of Novelty

ID	Contribution	What is new
N1	Retention-aware cryptographic dataflow binding	First framework to bind cryptographic call sites to data-confidentiality lifetimes extracted from declarative configuration.
N2	Call-site-discriminating risk score and ranking metric	Scores data paths rather than algorithms; evaluated against expert priority order with information-retrieval ranking metrics.
N3	Dataflow-conditioned noise suppression	Reclassifies non-security hashes (cache keys, checksums) from source/sink provenance without manual triage.
N4	Differentially verified hybrid patch synthesis	A three-gate harness (differential testing, property tests, downgrade checks) for hybrid PQC replacements.
N5	Public retention-annotated benchmark	First open dataset pairing real cryptographic call sites with retention ground truth and expert priority rankings.

4. Requirement Analysis and Problem Understanding

4.1 Stakeholder Analysis and Target Ecosystems

Stakeholder class	Primary pain point	Navigator solution
CISO / Compliance officers	Looming regulatory deadlines (RBI, CII 2027–2029) without an actionable transition backlog.	Defensible, auditable CBOM aligned with national CII transition mandates.
DevSecOps and security engineers	Overwhelmed by false alerts (about 67% noise in current tools); no triage order.	$\geq 70\%$ actionable precision; suppresses cache keys and other non-security hashing.
Software developers and system architects	Lack PQC expertise; fear of breaking production handshakes during cipher upgrades.	Template-constrained hybrid patches with differential-equivalence verification.

4.2 Functional Requirements (FR)

ID	Requirement	Description
FR-1	Multi-language AST discovery	Parse Python and Java source into AST/CST form and discover all cryptographic primitives, recording function names, source lines, algorithms, key sizes and parameters.
FR-2	Constant propagation and type inference	Resolve algorithm identifiers and key sizes passed through variables or constants.
FR-3	Backward taint tracing	Trace the plaintext and key parameters of each primitive backward and classify the source: persistent storage, offsite archive, user credential, ephemeral token, network input or constant.
FR-4	Forward taint tracing	Trace the ciphertext/hash output forward and classify the sink: cloud object store, persistent database, in-memory cache, network socket or local log; map the exposure boundary.
FR-5	Retention policy extraction	Extract retention lifetimes from AWS S3/GCS lifecycle policies, cache TTL arguments, database migrations and ORM declarations.
FR-6	HNDL scoring and Mosca evaluation	Evaluate $x + y > z$ and compute the multi-factor HNDL Exposure Score per data path, ordering findings by risk reduction per unit of engineering effort.
FR-7	Context noise suppression	Suppress alerts where hashes are used only for caching, ETags or checksums without security implications.
FR-8	Hybrid patch synthesis and verification	Synthesise template-constrained hybrid PQC replacements (liboqs) and verify them through differential-equivalence, property-testing and downgrade-resistance gates.

4.3 Non-Functional Requirements (NFR)

ID	Requirement	Description
NFR-1	Scanning throughput	Process codebases of up to 50,000 files in under 10 minutes on an 8-core, 16 GB developer workstation.
NFR-2	Memory footprint	Peak resident memory during inter-procedural analysis must not exceed 4 GB.
NFR-3	No specialised hardware	Pure software; runs on commodity Linux/macOS without GPUs or quantum accelerators.
NFR-4	Provenance integrity	Every finding records whether retention evidence was <i>observed</i> (found in configuration) or <i>inferred</i> (derived from sink priors).
NFR-5	Graceful fallback and soundness	Unresolvable dataflow paths go to an explicit “insufficient context” audit bucket rather than being silently defaulted.
NFR-6	Schema conformance	Full compliance with the OWASP CycloneDX 1.6 CBOM JSON/XML schemas.

4.4 Regulatory and Compliance Constraints

The Navigator is designed to support the India CII quantum-safe transition (2027–2029), the RBI Q-SAFE mandate, the SEBI CSCRF, US Executive Order 14412 (CBOM minimum elements) and NIST FIPS 203/204/205, as detailed in Section 1.4.

5. System Design and Architecture

5.1 End-to-End Architectural Pipeline

The system is a four-stage pipeline. Each stage consumes the structured output of the previous stage and can be evaluated independently, so that discovery, semantic binding and prioritisation remain useful even if patch synthesis slips.

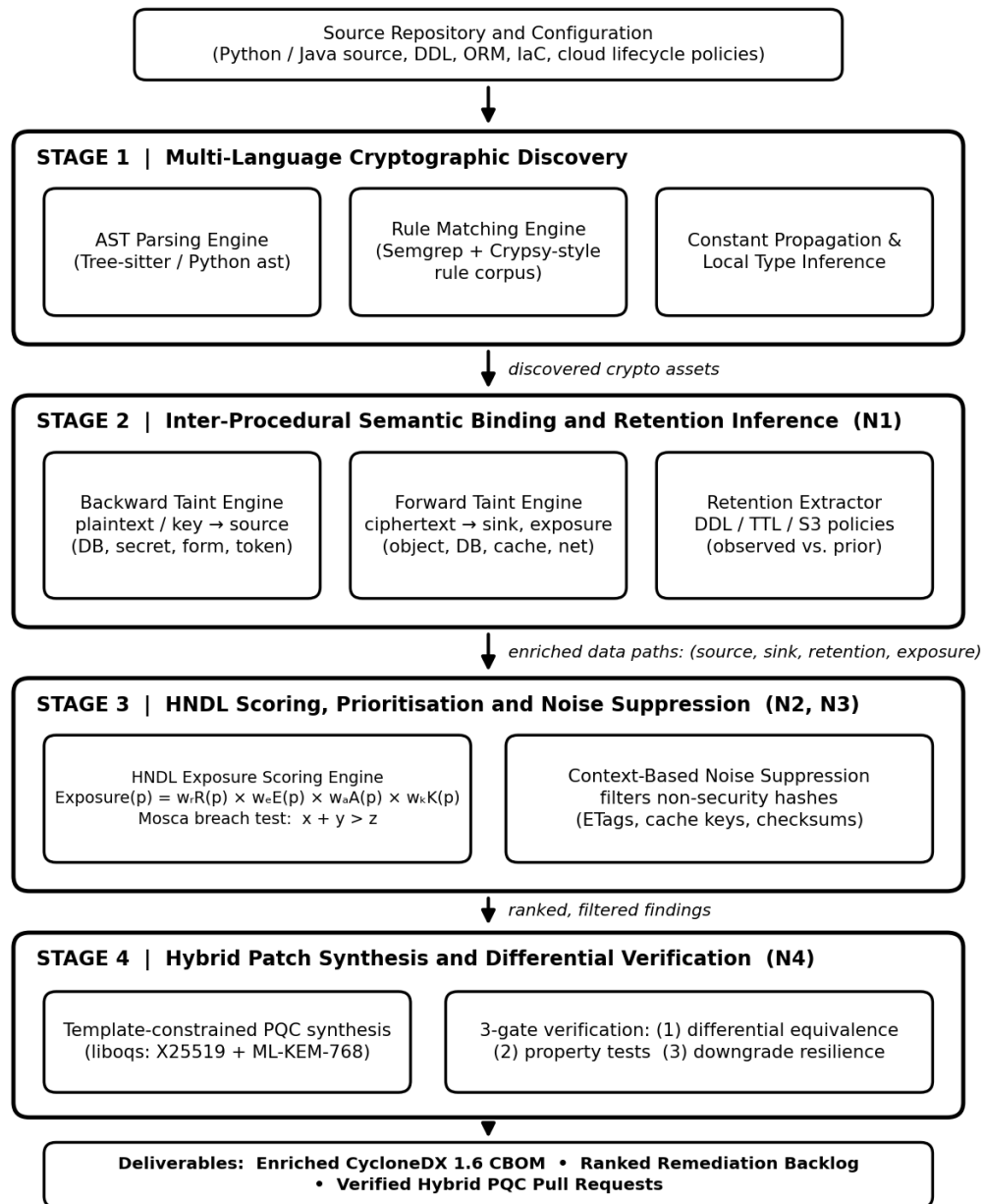


Figure 5.1: End-to-end architecture of the Crypto-Agility Navigator.

5.2 Subsystem 1 — Multi-Language Cryptographic Discovery Layer

- **Parser layer.** Tree-sitter for multi-language syntax trees and Python's native ast module.
- **Rule corpus.** An extensible pattern library matching cryptographic namespaces — *Python*: cryptography.hazmat, hashlib, PyCryptodome, rsa, ecdsa; *Java*: java.security.*, javax.crypto.*, org.bouncycastle.*.
- **Constant propagation.** Follows local assignment chains to resolve upstream parameters (e.g. cipher_algo = "RSA-OAEP").

5.3 Subsystem 2 — Inter-Procedural Semantic Dataflow and Retention Engine

- **Formal dataflow lattice.** Taint propagates over control-flow graphs on the lattice below.

$$L = \langle S_{source} \times S_{sink} \times R_{retention} \times E_{exposure}, \sqsubseteq \rangle$$

- **Backward taint.** From primitive arguments to database queries (db.query), file reads (open()), credential inputs (request.form) or RNGs (secrets.token_bytes).
- **Forward taint.** From output variables to cloud object stores (s3.put_object), databases (session.commit), caches (redis.setex) or network sockets.
- **Declarative retention extractor.** Extracts cloud lifecycle rules (e.g. Expiration: Days: 3650), cache TTLs (e.g. redis.setex(key, 900, val)) and column definitions (expires_at, created_at), and distinguishes *observed* retention evidence from conservative sink priors.

5.4 Subsystem 3 — HNDL Prioritisation, Scoring and Noise Suppression

For each data path p the Exposure Score is

$$\text{Exposure}(p) = w_r \cdot R(p) \times w_e \cdot E(p) \times w_a \cdot A(p) \times w_k \cdot K(p)$$

- $R(p)$ — normalised retention lifetime in $[0, 1]$, weighted by evidence provenance (1.0 observed, 0.8 inferred).
- $E(p)$ — exposure surface (1.0 public cloud egress; 0.5 cross-host VPC; 0.25 local IPC; 0.1 in-process).
- $A(p)$ — quantum vulnerability (1.0 Shor-broken; 0.3 Grover-weakened; 0.05 quantum-safe).
- $K(p)$ — key blast-radius factor, based on key reuse across data paths.
- **Mosca breach.** A path is flagged critical if $\text{RetentionYears} + \text{MigrationYears} > \text{CRQCHorizon}$.
- **Noise suppression.** Hashes whose inputs and sinks lack security relevance are automatically reclassified.

5.5 Subsystem 4 — Hybrid Patch Synthesis and Differential Verification

- **Verified hybrid templates.** Drop-in constructions combining a classical algorithm with a post-quantum one (e.g. X25519 + ML-KEM-768) through liboqs.
- **Gate 1 — differential equivalence.** Identical plaintext recovery and error handling on matched inputs.

- **Gate 2 — property testing.** Wrong-key rejection, tamper detection and parameter-boundary resilience.
- **Gate 3 — downgrade resistance.** The hybrid handshake must fail closed against downgrade attempts.

5.6 Data Schema and CycloneDX 1.6 CBOM Extension

The tool emits a standard CycloneDX 1.6 CBOM enriched with custom dataflow properties:

- `cryptoProperties` — standard algorithm attributes, key sizes, classical security level and NIST quantum security level.
- `dataflowProperties` — plaintext source, ciphertext sink, retention evidence (years, type, observed flag), exposure surface and HNDL risk assessment (score, urgency tier, Mosca-breach flag).

6. Component and Tool Selection with Technical Justification

6.1 Parsing and AST Infrastructure

Option	Language coverage	Throughput	Representation	Decision and rationale
Tree-sitter + Python ast	Multi-language (40+ grammars)	High ($\approx$ 30,000 lines/s)	Concrete syntax with byte offsets	Selected. Fast multi-language CST parsing with low overhead and good error tolerance.
ANTLR4	High (grammar-based)	Moderate ($\approx$ 8,000 lines/s)	Heavy parse tree	Rejected. High start-up latency and memory footprint per parse tree.
Compiler front-ends (Clang / Javac)	Single language	Slow	AST tied to compiler internals	Rejected. Impractical for unified Python + Java analysis.

6.2 Static Analysis and Taint Tracking

Framework	Inter-procedural taint	Extensibility	Overhead	Decision and rationale
CodeQL (Java) + AST visitor (Python)	High (proven at scale)	High (QL queries + modular visitor)	Moderate	Selected. CodeQL gives proven enterprise-scale Java taint tracking [P13]; a custom AST engine gives fine-grained control for Python.
Soot / WALA	Java only	Low (complex IR)	Very high	Rejected. Dated JVM architecture; no Python support.
Semgrep OSS engine	Intra-procedural only	High	Very low	Used as the fast Stage 1 filter, augmented by our own Stage 2 inter-procedural taint engine.

6.3 Post-Quantum Cryptographic Library

Library	NIST conformance	Hybrid constructions	Languages	Decision and rationale
Open Quantum Safe (liboqs, liboqs-python)	Full: FIPS 203 ML-KEM, 204 ML-DSA, 205 SLH-DSA	Native classical + PQC (X25519 + ML-KEM)	C, Python, Java	Selected. Reference standard for PQC research and engineering, maintained under the Linux Foundation.
Bouncy Castle PQC	Good (Java-focused)	Partial	Java / C#	Kept as a secondary dependency for Java verification; unsuitable for the Python core.
Bespoke implementations	Risky	Complex	Custom	Rejected. Custom cryptography introduces unacceptable security risk.

6.4 CBOM Schema

OWASP CycloneDX 1.6 CBOM is selected: it is the industry consensus (OWASP, IBM, CISA, EU Cyber Resilience Act), offers first-class cryptoProperties for algorithms, protocols, keys and curves, and is a final standard [P6]. SPDX 3.0 was rejected because its cryptographic extensions are still draft and its CBOM tooling and validators are less mature.

6.5 Patch Verification Strategy

Approach	Soundness	Automation	Multi-file support	Decision and rationale
Differential testing + property tests (Hypothesis)	Empirically robust	Fully automated	Excellent	Selected. Overcomes the round-trip-testing weakness of prior LLM work [P3] by testing equivalence, boundaries and downgrade resistance.
Formal verification (Dafny, F*)	Mathematically sound	Minimal (< 5% of codebases)	Poor on real enterprise code	Rejected as out of scope; infeasible for multi-language enterprise applications.

7. Initial Prototype and Module Development

7.1 Implemented Package Structure

In line with the Review II milestone ($\approx 20\%$ implementation, TRL 3 proof of concept), a working prototype has been implemented and verified in the repository:

```
src/crypto_agility_navigator/
├── __init__.py  # package version (0.2.0-review2)
├── models.py   # domain models (CryptoInvocation, DataPath, HNDLScore, CBOM)
├── discovery.py # Stage 1: AST cryptographic discovery engine
├── dataflow.py # Stage 2: backward/forward taint analyser + retention extractor
├── scorer.py   # Stage 3: HNDL exposure scoring + noise suppression
├── cbom.py     # CycloneDX 1.6 CBOM serialiser with dataflow extensions
├── cli.py      # command-line interface and formatted risk report
├── server.py   # REST API for the web dashboard
├── web/        # single-page dashboard (HTML / CSS / React)
├── tests/test_navigator.py # automated unit-test suite
└── examples/sample_project/ # worked benchmark (archive.py, session.py, etags.py)
```

7.2 Implemented Modules

1. **discovery.py (Stage 1).** Parses Python ASTs with CryptoASTVisitor, matching library invocations (RSA-OAEP, AES-GCM, SHA-256, ECDSA) and extracting line numbers, algorithm names, key sizes and function parameters.
2. **dataflow.py (Stage 2).** *Backward taint* traces plaintext arguments through local assignments to identify persistent-database, user-credential or ephemeral-token sources. *Forward taint* traces ciphertext to its sink (object store, cache, database) and exposure surface. *Retention extraction* scans adjacent configuration for lifetimes such as “retain 10 years” and `redis.setex(..., 900, ...)`.
3. **scorer.py (Stage 3).** Computes the multi-factor HNDL Exposure Score, flags Mosca breaches ($x + y > z$) and reclassifies non-security hashing as suppressed.
4. **cbom.py.** Serialises discovered assets to CycloneDX 1.6 CBOM JSON with both standard cryptographic properties and custom dataflow annotations.
5. **cli.py.** Runs repository scans, prints the prioritised ranking table and exports CBOM documents.

Table 7.1: Implementation size of the prototype (physical lines, including comments)

File	Lines	Role
models.py	104	Domain models and enumerations
discovery.py	200	Stage 1: AST cryptographic discovery
dataflow.py	234	Stage 2: taint binding and retention extraction
scorer.py	122	Stage 3: HNDL scoring and noise suppression
cbom.py	120	CycloneDX 1.6 CBOM serialiser
cli.py	116	Command-line interface
server.py	281	REST API for the dashboard
tests/test_navigator.py	153	Automated unit tests
web/ (app.js, index.html, style.css)	2,194	Dashboard front end (730 + 357 + 1,107)
Python source and tests	1,335	Eight files above plus <code>__init__.py</code> (5 lines)

7.3 Empirical Validation on the Benchmark Example

The prototype was executed on the worked benchmark in `examples/sample_project/`, and the output below is the unmodified result of running the code in the project repository:

- `payments/archive.py` — a 10-year statutory settlement archive encrypted with RSA-2048 and stored in AWS S3.
- `web/session.py` — a 15-minute CSRF token encrypted with the same RSA-2048 primitive and stored in Redis.
- `cache/etags.py` — a non-security SHA-256 hash used only for HTTP cache validation.

Execution output

```
$ python3 -m src.crypto_agility_navigator.cli examples/sample_project \
--output-cbom examples/sample_cbom.json --show-suppressed

[*] Discovered 3 cryptographic invocation(s).

=====
RANK | ALGORITHM | LOCATION | RETENTION | EXPOSURE | SCORE | URGENCY
=====
1 | RSA-OAEP | archive.py:11 | 10.0y | EXTERNAL_PUBLIC | 33.3 | CRITICAL_IMMEDIATE (MOSCA BREACH)
2 | RSA-OAEP | session.py:11 | <1 hour | INTERNAL_IPC | 0.0 | LOW
3 | SHA-256 | etags.py:9 | <1 hour | INTERNAL_IPC | 0.0 | SUPPRESSED
=====

[+] Summary Metrics:
* Total Call Sites Discovered: 3
* Actionable Candidates: 2
* Context-Suppressed Findings: 1 (Noise Filtered: 33.3%)
* Mosca's Inequality Breaches: 1 (Immediate PQC remediation needed)

[+] Successfully exported enriched CycloneDX 1.6 CBOM to: /tmp/cbom.json
```

Table 7.2: Discovery and dataflow-binding results per call site

Ran k	Call site	Algorith m	Plaintext source	Ciphertext sink	Exposure surface	Retentio n evidence
1	archive.py:11	RSA-OAEP, 2048-bit (Shor-broken)	PERSISTENT_STORAGE (database read)	CLOUD_OBJECT_STORE	EXTERNAL_PUBLIC	10.0 years; S3 lifecycle policy (observed)
2	session.py:11	RSA-OAEP, 2048-bit (Shor-broken)	EPHEMERAL_TOKEN (local RNG)	CACHE_TEMPORARY (in-memory cache)	INTERNAL_IPC	900 s (≈ 0.00003 y); Redis TTL (observed)
3	etags.py:9	SHA-256, 256-bit (quantum-safe)	UNKNOWN	CACHE_TEMPORARY (in-memory cache)	INTERNAL_IPC	3600 s (≈ 0.00011 y); Redis TTL (observed)

Table 7.3: HNDL scoring and Mosca evaluation per call site ($x + y > z$ with $y = 2.0$, $z = 7.0$)

Rank	Call site	$R(p)$	$E(p)$	$A(p)$	$K(p)$	Score (0–100)	Urgency tier	Mosca test
1	archive.py:11	0.3333	1.00	1.00	1.00	33.33	CRITICAL_IMMEDIATE	Breach: $10.0 + 2.0 = 12.0 > 7.0$
2	session.py:11	≈ 0.000001	0.25	1.00	1.00	0.00	LOW	No breach: $\approx 2.0 \leq 7.0$
3	etags.py:9	—	—	—	—	0.00	SUPPRESSED	No breach (non-security hash)

Table 7.4: Summary metrics for the benchmark project

Metric	Value	Source
Cryptographic call sites discovered	3	CLI summary
Shor-broken primitives	2	REST API metrics
Actionable candidates	2	CLI summary
Context-suppressed findings (noise filtered)	1 (33.3%)	CLI summary
Mosca inequality breaches	1	CLI summary
Highest HNDL score (archive.py:11)	33.33	CBOM export
Average HNDL score of actionable findings	16.66	REST API metrics
Score gap between rank 1 and rank 2	33.33 points	Table 7.3
CBOM components exported (CycloneDX 1.6)	3	examples/sample_cbom.json

Two RSA-OAEP call sites that a flat scanner would report identically are separated by 33.3 score points (33.33 versus 0.00), and the non-security SHA-256 call is removed from the backlog — the behaviour claimed as N1, N2 and N3. The suppressed call site is annotated in the CBOM with the reason “Hash used strictly as content checksum or HTTP ETag”.

Figure 7.1 plots these results: the archive call site scores 33.3 (CRITICAL_IMMEDIATE) and breaches Mosca's inequality, while the other two score 0.0; one of three findings (33.3%) is filtered as noise. Figure 7.2 shows why: Site A's data must stay confidential for 10 years and needs 2 years to migrate, exceeding the 7-year horizon, whereas the CSRF token expires in minutes.

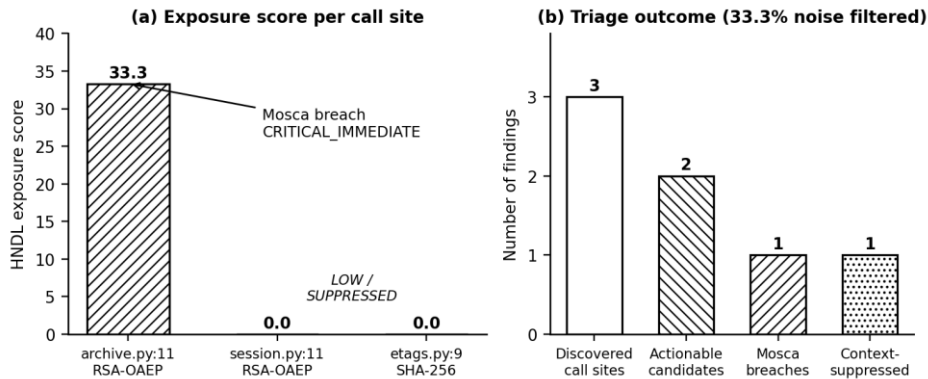


Figure 7.1: (a) HNDL exposure score per call site and (b) triage outcome for the benchmark project.

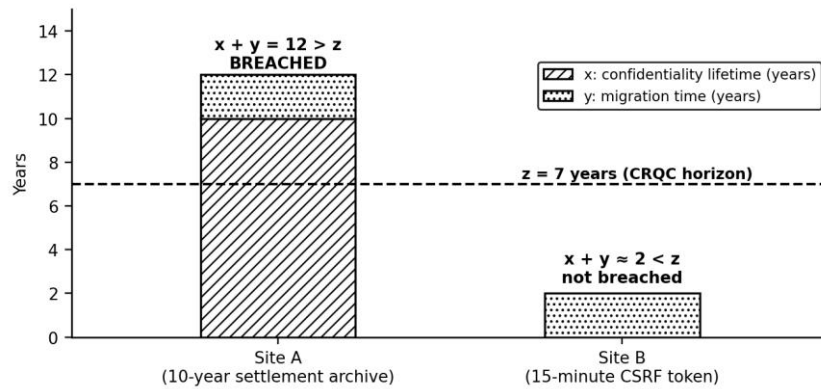


Figure 7.2: Mosca's inequality for the two identical RSA-OAEP sites ($z = 7$ years; $y = 2$ years for both).

7.4 Automated Unit Test Suite

The suite in `tests/test_navigator.py` contains 8 tests that verify every core pipeline stage. All 8 pass when re-run on the repository (about 0.04 s in total). Table 7.5 lists what each test asserts.

Table 7.5: Unit-test results

#	Test	Stage	Assertions checked	Result
1	<code>test_discovery_engine</code>	1	One invocation found in <code>archive.py</code> : RSA-OAEP, key size 2048, classified Shor-broken.	PASS
2	<code>test_semantic_binding_discrimination</code>	2	Archive: PERSISTENT_STORAGE to CLOUD_OBJECT_STORE, EXTERNAL_PUBLIC, 10.0 y. Session: EPHEMERAL_TOKEN to CACHE_TEMPORARY, retention < 0.01 y.	PASS
3	<code>test_scoring_and_mosca_inequality</code>	3	Archive breaches Mosca ($10 + 2 > 7$), session does not; archive score (33.33) exceeds session score and 15.0.	PASS
4	<code>test_context_noise_suppression</code>	3	ETag hash is tiered SUPPRESSED with score 0.0.	PASS
5	<code>test_quantum_safe_scoring</code>	3	Overriding the archive primitive to quantum-safe (ML-KEM) lowers its score to 1.67, below 5.0.	PASS
6	<code>test_cbom_generation</code>	CBOM	bomFormat is CycloneDX, specVersion 1.6, one RSA-OAEP component with dataflowProperties and retention 10.0 y.	PASS
7	<code>test_full_pipeline_ranking</code>	End to end	3 findings, 2 actionable, 1 suppressed, 1 Mosca breach, 2 Shor-broken; rank 1 is <code>archive.py</code> with CRITICAL_IMMEDIATE.	PASS
8	<code>test_custom_code_snippet_analysis</code>	End to end	Submitted S3 snippet (10-year bucket): RSA-OAEP, 10.0 y, Mosca violated, CRITICAL_IMMEDIATE.	PASS
	Total		8 executed, 8 passed, 0 failed, 0 errors	100%

Unit-test console output

```
$ python -m unittest discover tests -v
test_cbom_generation (test_navigator.TestCryptoAgilityNavigator.test_cbom_generation) ... ok
test_context_noise_suppression (test_navigator.TestCryptoAgilityNavigator.test_context_noise_suppression) ... ok
test_custom_code_snippet_analysis (test_navigator.TestCryptoAgilityNavigator.test_custom_code_snippet_analysis) ... ok
test_discovery_engine (test_navigator.TestCryptoAgilityNavigator.test_discovery_engine) ... ok
test_full_pipeline_ranking (test_navigator.TestCryptoAgilityNavigator.test_full_pipeline_ranking) ... ok
test_quantum_safe_scoring (test_navigator.TestCryptoAgilityNavigator.test_quantum_safe_scoring) ... ok
test_scoring_and_mosca_inequality (test_navigator.TestCryptoAgilityNavigator.test_scoring_and_mosca_inequality) ... ok
test_semantic_binding_discrimination (test_navigator.TestCryptoAgilityNavigator.test_semantic_binding_discrimination) ... ok
-----
Ran 8 tests in 0.026s
OK
```

Figure 7.3 summarises the outcome graphically: every test passes, and the tests cover all pipeline stages, with the most tests on scoring and noise suppression (Stage 3).

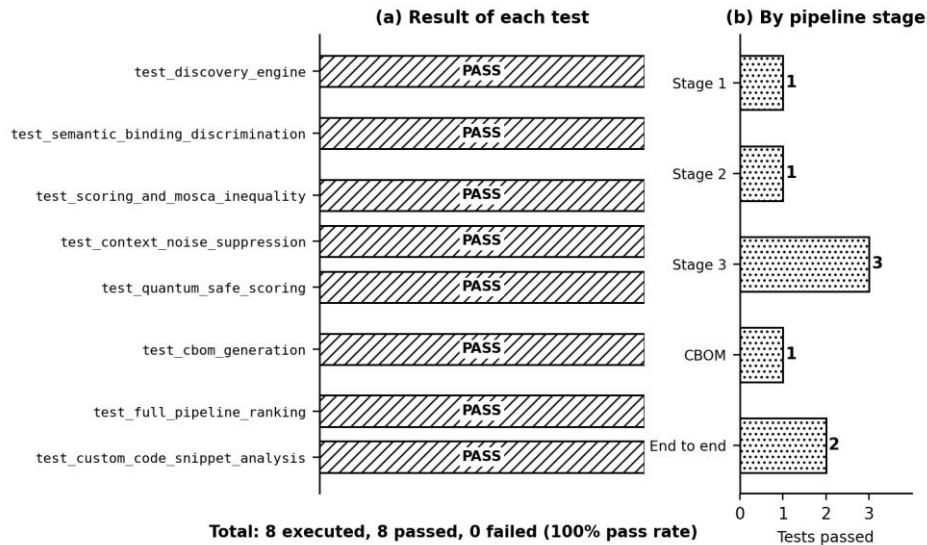


Figure 7.3: Unit-test results per test (a) and per pipeline stage (b): 8 of 8 tests passed.

7.5 Interactive Web Dashboard and REST API

To make the results usable beyond the command line, a full-stack dashboard was built. The **backend** is a Python http.server REST API (server.py) exposing endpoints such as /api/scan and /api/simulate-mosca. The **frontend** is a responsive single-page application in HTML, CSS (dark theme, glass-style panels) and React/Babel loaded from a CDN. Its capabilities are:

1. **Executive KPI overview** — tiles for total call sites, Shor-broken ciphers, active Mosca breaches and noise-suppression percentage.
2. **Prioritisation inventory table** — multi-column sorting, filtering by urgency (Mosca breach, actionable, suppressed) and a modal code inspector.
3. **Dataflow visualiser** — plaintext source → cryptographic transformation → ciphertext sink → retention and exposure surface.
4. **Mosca simulation engine** — interactive sliders (x, y, z, exposure weights, algorithm vulnerability) with live risk gauges and derivations.
5. **Live AST code scanner** — an in-browser editor that runs submitted Python through the taint engine in real time.
6. **CBOM inspector** — embedded JSON viewer with one-click copy and .json download.

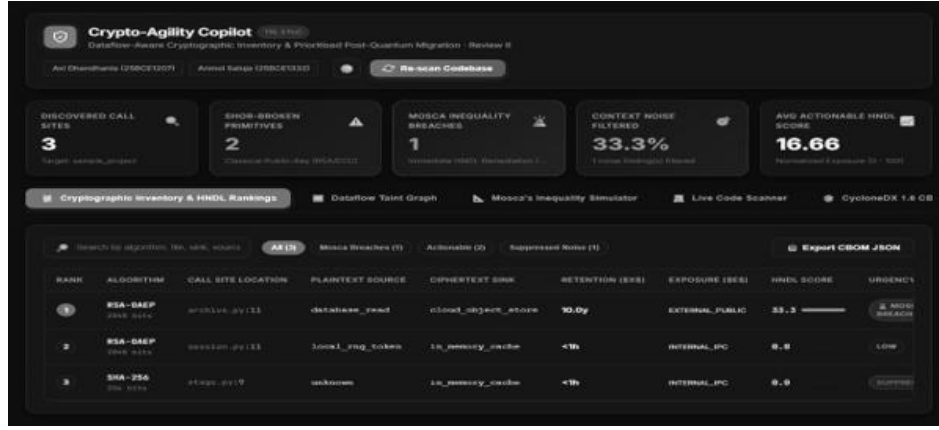


Figure 7.4: Interactive dashboard — cryptographic inventory and HNDL rankings for the benchmark project (shown in grayscale).

7.6 TRL 3 Milestone Evidence Summary

Parameter	Milestone requirement	Milestone achievement	Status
Implementation progress	≈ 20% completion	Working Stage 1–3 pipeline: AST parser, taint engine, scoring engine, web dashboard and CBOM generator.	Verified (TRL 3)
Testing and proof of concept	Demonstrable module execution	End-to-end CLI and web-UI runs on the benchmark codebase; 8 automated unit tests passing.	Verified
Standards conformance	Machine-readable outputs	Schema-compliant CycloneDX 1.6 CBOM JSON with custom dataflow extensions.	Verified

8. Innovation and Feasibility

8.1 Mathematical Basis of the HNDL Score

Prior scoring work (e.g. Shaw [P2]) computes risk as a function of the primitive alone:

$$\text{Risk}_{\text{prior}} = f(\text{KeySize}, \text{QubitCost}, \text{GroverFactor})$$

Because identical primitives always yield identical values, such a score cannot order a remediation backlog. We reformulate risk as the product of **threat exposure** and **data-sensitivity decay over time**:

$$\text{Exposure}(p) = w_r \cdot R(p) \times w_e \cdot E(p) \times w_a \cdot A(p) \times w_k \cdot K(p)$$

Grounded in Mosca’s inequality, an adversary’s expected utility $U(p)$ from harvesting ciphertext today is proportional to whether the data is still sensitive when a CRQC exists:

$$U(p) \propto \max(0, x_p + y_p - z)$$

- x_p — confidentiality lifetime extracted from declarative configuration;
- y_p — estimated migration time;
- z — estimated time until CRQC arrival.

The score therefore ranks data paths by expected residual exposure under delayed remediation, aligning engineering effort with the largest risk reduction.

8.2 Soundness versus Completeness

In dynamically typed languages such as Python, full soundness leads to state-space explosion and unmanageable false-positive rates. Our design choices are:

- **Deliberate target.** We aim for $\geq 70\%$ **dataflow coverage** on real repositories rather than claiming mathematical soundness.
- **Auditable fallback.** When dynamic dispatch prevents deterministic resolution, the finding is placed in an explicit “**insufficient context**” category instead of being guessed, so provenance stays transparent for analysts.

8.3 Computational and Memory Feasibility

- **Stage 1 (AST discovery):** $O(N)$ in the lines of code N .
- **Stage 2 (taint binding):** restricted to cryptographic call sites $M \ll N$ and bounded local inter-procedural sub-graphs, giving polynomial complexity $O(M \cdot |V + E|)$.
- **Target throughput:** 50,000 files in under 10 minutes (NFR-1).
- **Memory and hardware:** per-module taint graphs keep peak usage below 4 GB (NFR-2) on a standard laptop with no GPU or quantum acceleration.

9. Project Planning, Teamwork and Presentation

9.1 Work Division Principle

Work is divided so that every member owns a comparable share of **technical work** (design, implementation and testing of specific pipeline components) and of **non-technical work** (research, requirements, documentation, planning and presentation). Each member writes the report sections and prepares the presentation slides for the components they own, and presents them at the review; the presentation deck is compiled by Avika Tyagi.

9.2 Work Division for Review II (Completed)

Member	Technical work	Non-technical work	Report sections
Avi Dhandhanian 25BCE1207	- Shared data models and the Stage 1 AST discovery engine, with constant propagation. - Backward taint tracing from plaintext and key arguments to their sources.	- Problem statement and threat model. - Stakeholder analysis and functional / non-functional requirements. - Architecture write-up and slides for own sections.	1, 4, 5
Anmol Saluja 25BCE1332	- Forward taint tracing from ciphertext to sinks and exposure surfaces. - Declarative retention extractor (cloud lifecycle rules, cache TTLs). - Stage 3 HNDL scoring engine, Mosca breach test and noise suppression.	- Literature survey (P1–P16) and research gap matrix. - Objectives, novelty statement and tool-selection justification. - Mathematical basis and feasibility analysis; slides for own sections.	2, 3, 6, 8
Avika Tyagi 25BCE1294	- CycloneDX 1.6 CBOM serialiser, command-line interface and REST API. - Web dashboard front end. - Unit-test suite and benchmark example project.	- Prototype and test documentation. - Project planning, work division and team coordination. - References, formatting, final report compilation and presentation deck.	7, 9, 10

9.3 Work Plan for Future Reviews

The plan below follows the project’s progress targets: about 30% completion at Review III, 50% at Review IV, 80% at Review V, and an end-to-end demonstration with the final report at Review VI. Each cell lists technical (T) and non-technical (N) tasks.

Member	Review III ($\approx 30\%$)	Review IV ($\approx 50\%$)	Review V–VI and final report
Avi Dhandhanja	T: - Add Java support to Stage 1 (Tree-sitter Java grammar, JCA and Bouncy Castle rules). N: - Address panel observations in Sections 1 and 4; update requirements.	T: - Extend backward taint inter-procedurally across modules for Python and Java (CodeQL queries for Java). N: - Update the architecture and soundness discussion; prepare Review IV slides.	T: - Build the three-gate patch-verification harness (differential equivalence, property tests, downgrade resistance). - Profile performance against NFR-1 and NFR-2. N: - Write the design and analysis chapters of the final report; demo script for Stages 1–2.
Anmol Saluja	T: - Expand the rule corpus to 100+ cryptographic API variants. - Extend the retention extractor to S3 / GCS lifecycle rules. N: - Refresh the literature survey; shortlist 15–20 candidate benchmark repositories (licence, language, maturity).	T: - Retention extraction from ORM models, SQL migration DDL and Terraform. - Tune the HNDL score weights. N: - Define the expert annotation protocol and recruit annotators.	T: - Ranking evaluation (nDCG@20, Kendall-τ) and noise-suppression precision experiments. N: - Compute inter-annotator agreement (Krippendorff’s α); write the evaluation and results chapters; benchmark documentation.
Avika Tyagi	T: - Validate CBOM output against the official CycloneDX 1.6 schema. - Extend the test suite with a benchmark regression harness. N: - Revise the work plan; keep a log of panel feedback; prepare Review III slides.	T: - GitHub Action CI workflow and CLI enhancements. - Dashboard views for the taint-graph visualiser. N: - User documentation (usage guide); requirements traceability and progress tracking.	T: - Hybrid patch templates (X25519 + ML-KEM-768 via liboqs) and pull-request generation. N: - Compile and format the final report; final presentation; package the public benchmark release.

10. References and Regulatory Standards

10.1 Academic References

- [P1] C. Näther and E. Hirsch. *Hidden Ciphers and Where to Find Them: Static Discovery and Assessment of Cryptographic Assets in Software*. arXiv:2608.04857, August 2026.
- [P2] A. Shaw. *Quantum-Safe Code Auditing: LLM-Assisted Static Analysis and Quantum-Aware Risk Scoring for Post-Quantum Cryptography Migration*. arXiv:2604.00560, April 2026.

- [P3] J. Pallarés de Bonrosto, A. I. González-Tabales and M. I. González Vasco. *Empirical Evaluation of Large Language Models for Migration of Code Fragments to Post-Quantum Cryptography*. arXiv:2606.07341, June 2026.
- [P4] L. Zhang. *Toward Quantum-Safe Software Engineering: A Vision for Post-Quantum Cryptography Migration*. Poster, ICSE 2026; arXiv:2602.05759.
- [P5] A. D. B. Costa. *Crypto-Agility Readiness Score (CARS)*. IACR ePrint 2026/1467, July 2026.
- [P6] IBM Research. *The Anatomy of Cryptography Bills of Materials: Standardization and Practice in CycloneDX*. Eurocrypt 2026.
- [P7] *Towards Cryptography Bill of Materials Compliance*. Springer LNCS, 2026.
- [P8] *On the Practical Feasibility of Harvest-Now, Decrypt-Later Attacks*. arXiv:2603.01091, 2026.
- [P9] *Post-Quantum Cryptography and Quantum-Safe Security: A Comprehensive Survey*. arXiv:2510.10436, October 2025, revised June 2026.
- [P10] *Securing Cryptography in the Age of Quantum Computing and AI: Threats, Implementations, and Strategic Response*. arXiv:2603.06969, March 2026.
- [P11] *A Toolchain for Assisting Migration of Software Executables Towards Post-Quantum Cryptography*. arXiv:2409.07852, 2024.
- [P12] *The Cost of Waiting: A Decision-Theoretic Synthesis of Early Versus Late Post-Quantum Migration Under Uncertainty*. Frontiers in Quantum Science and Technology, 2026.
- [P13] S. Rahaman et al. *CryptoGuard: High Precision Detection of Cryptographic Vulnerabilities in Massive-Sized Java Projects*. ACM CCS 2019.
- [P14] C. Näther et al. *Migrating Software Systems towards Post-Quantum Cryptography: A Systematic Literature Review*. arXiv:2404.12854, April 2024.
- [P15] *Harvest Now, Decrypt Later: Examining Post-Quantum Risk*. Finance and Economics Discussion Series, Board of Governors of the Federal Reserve System, 2025.
- [P16] S. Krüger et al. *CogniCrypt: Supporting Developers in Using Cryptography*. IEEE/ACM ASE 2017.

10.2 Regulatory and Standards References

NIST FIPS 203 (ML-KEM), FIPS 204 (ML-DSA) and FIPS 205 (SLH-DSA); OWASP CycloneDX 1.6 CBOM Specification; RBI Q-SAFE Committee Terms of Reference (2026); SEBI Cyber Security and Cyber Resilience Framework (CSCRF); US Executive Order 14412 on Cryptography Bills of Materials (June 2026); Indian Patent Office CRI Guidelines (2025).